



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 01

NOMBRE COMPLETO: PABLO SEBASTIAN NOYOLA
TORRES

N° de Cuenta: 320023213

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 06

SEMESTRE 2027 - 1

FECHA DE ENTREGA LÍMITE: 29 DE AGOSTO DE 2026

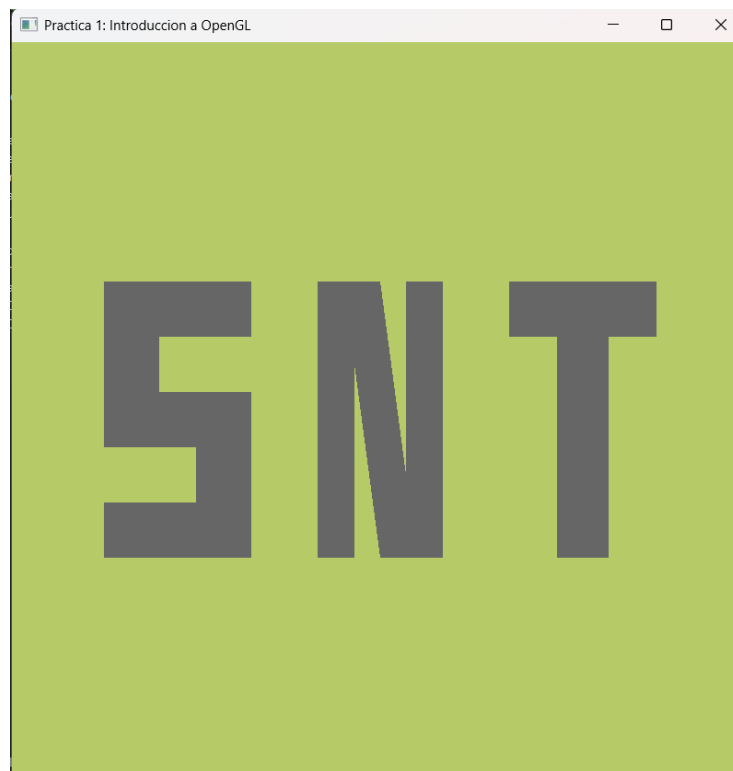
CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

1.- Ventana cambia el color de fondo de forma random tomando rango de colores RGB y con una periodicidad de 2 segundos. (Verificar que al ejecutar el programa varias veces el orden de los colores si lo vean aleatorio y no siempre los mismos)

Capturas de ejecución:





Código de ejercicio:

```
167 | srand((unsigned int)time(NULL));
170 | double ultimoTiempo = glfwGetTime();
171 | float r = (float)rand() / RAND_MAX;
172 | float g = (float)rand() / RAND_MAX;
173 | float b = (float)rand() / RAND_MAX;
179 | double tiempoActual = glfwGetTime();
180 |
181 | if (tiempoActual - ultimoTiempo >= 2.0) {
182 |     r = (float)rand() / RAND_MAX;
183 |     g = (float)rand() / RAND_MAX;
184 |     b = (float)rand() / RAND_MAX;
185 |     ultimoTiempo = tiempoActual;
186 | }
188 | // Limpiar la ventana
189 | glClearColor(r, g, b, 1.0f);
190 | glClear(GL_COLOR_BUFFER_BIT);
```

En este ejercicio implementé una lógica basada en temporizadores y números aleatorios en OpenGL para que el color de fondo de la ventana cambie de manera dinámica cada dos segundos. Para lograrlo, primero inicialicé una semilla con `srand` y `time` asegurando que la secuencia de colores sea distinta en cada ejecución, y declaré variables iniciales para el color RGB usando `rand() / RAND_MAX`. Después, dentro del bucle principal de renderizado, observe con `glfwGetTime` si ya habían transcurrido al menos 2 segundos desde el último cambio, de ser así, se actualizan las variables `r`, `g` y `b` con nuevos valores aleatorios y se reinicia el cronómetro. Finalmente, apliqué estos valores actualizados utilizando la función `glClearColor` para limpiar y poner color al fondo de la ventana de forma continua.

2.- 3 letras iniciales de sus nombres creadas a partir de triángulos, todas las letras son del mismo color.

Los dos ejercicios se muestran de forma simultánea y están en el mismo main

Capturas de ejecución:



Código de ejercicio:

```

31 void CrearTriangulo() {
32     GLfloat vertices[] = {
33         //PARA INICIALES SNT
34
35
36         // PARA S:
37         // Barra superior
38         -0.75f, 0.35f, 0.0f, -0.35f, 0.35f, 0.0f, -0.75f, 0.20f, 0.0f,
39         -0.35f, 0.35f, 0.0f, -0.35f, 0.20f, 0.0f, -0.75f, 0.20f, 0.0f,
40         // Bajada superior izquierda
41         -0.75f, 0.20f, 0.0f, -0.60f, 0.20f, 0.0f, -0.75f, 0.05f, 0.0f,
42         -0.60f, 0.20f, 0.0f, -0.60f, 0.05f, 0.0f, -0.75f, 0.05f, 0.0f,
43         // Barra central
44         -0.75f, 0.05f, 0.0f, -0.35f, 0.05f, 0.0f, -0.75f, -0.10f, 0.0f,
45         -0.35f, 0.05f, 0.0f, -0.35f, -0.10f, 0.0f, -0.75f, -0.10f, 0.0f,
46         // Bajada inferior derecha
47         -0.50f, -0.10f, 0.0f, -0.35f, -0.10f, 0.0f, -0.50f, -0.25f, 0.0f,
48         -0.35f, -0.10f, 0.0f, -0.35f, -0.25f, 0.0f, -0.50f, -0.25f, 0.0f,
49         // Barra inferior
50         -0.75f, -0.25f, 0.0f, -0.35f, -0.25f, 0.0f, -0.75f, -0.40f, 0.0f,
51         -0.35f, -0.25f, 0.0f, -0.35f, -0.40f, 0.0f, -0.75f, -0.40f, 0.0f,
52
53         // PARA N:
54         // Barra izquierda
55         -0.17f, 0.35f, 0.0f, -0.07f, 0.35f, 0.0f, -0.17f, -0.40f, 0.0f,
56         -0.07f, 0.35f, 0.0f, -0.07f, -0.40f, 0.0f, -0.17f, -0.40f, 0.0f,
57         // Barra diagonal del centro
58         -0.10f, 0.35f, 0.0f, 0.10f, -0.40f, 0.0f, 0.00f, 0.35f, 0.0f,
59         0.10f, -0.40f, 0.0f, -0.10f, 0.35f, 0.0f, 0.00f, -0.40f, 0.0f,
60         // Barra derecha
61         0.07f, 0.35f, 0.0f, 0.17f, 0.35f, 0.0f, 0.07f, -0.40f, 0.0f,
62         0.17f, 0.35f, 0.0f, 0.17f, -0.40f, 0.0f, 0.07f, -0.40f, 0.0f,
63
64
65         // PARA T:
66         // Barra horizontal superior
67         0.35f, 0.35f, 0.0f, 0.75f, 0.35f, 0.0f, 0.35f, 0.20f, 0.0f,
68         0.75f, 0.35f, 0.0f, 0.75f, 0.20f, 0.0f, 0.35f, 0.20f, 0.0f,
69         // Barra vertical central
70         0.48f, 0.20f, 0.0f, 0.62f, 0.20f, 0.0f, 0.48f, -0.40f, 0.0f,
71         0.62f, 0.20f, 0.0f, 0.62f, -0.40f, 0.0f, 0.48f, -0.40f, 0.0f
72     };

```

```

23 static const char* fShader = "
24 #version 330
25 out vec4 color;
26 void main()
27 {
28     color = vec4(0.4f, 0.4f, 0.4f, 0.4f);
29 };

```

```

192 // Para dibujar iniciales
193 glUseProgram(shader);
194 glBindVertexArray(VAO);
195
196 glDrawArrays(GL_TRIANGLES, 0, 90);
197
198 glBindVertexArray(0);
199 glUseProgram(0);

```

En este ejercicio implementé la geometría para dibujar las tres iniciales de mis nombres (S, N y T) utilizando una serie de vértices organizados en triángulos dentro de la función `CrearTriangulo()`, sumando un total de 90 vértices procesados mediante un VAO y VBO. Para asegurar que todas las letras compartieran exactamente el mismo aspecto, configuré un Fragment Shader fijo que les otorgara un color igual con el vector `vec4(0.4f, 0.4f, 0.4f, 0.4f)`. Finalmente, utilicé la función `glDrawArrays(GL_TRIANGLES, 0, 90)` dentro del bucle principal de renderizado para mandar a dibujar las letras en la pantalla.

2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

No se presentaron problemas en esta práctica.

3.- Conclusión:

Tras la realización de esta práctica, pude reafirmar y mejorar mis conocimientos en OpenGL, mejorando mi dominio en la gestión de buffers, la configuración de shaders y el manejo de transformaciones y tiempos con GLFW para crear gráficos interactivos.

1. Bibliografía en formato APA

TylerMSFT. (s. f.). Microsoft Learn.

<https://learn.microsoft.com/es-es/cpp/standard-library/random?view=msvc-170>